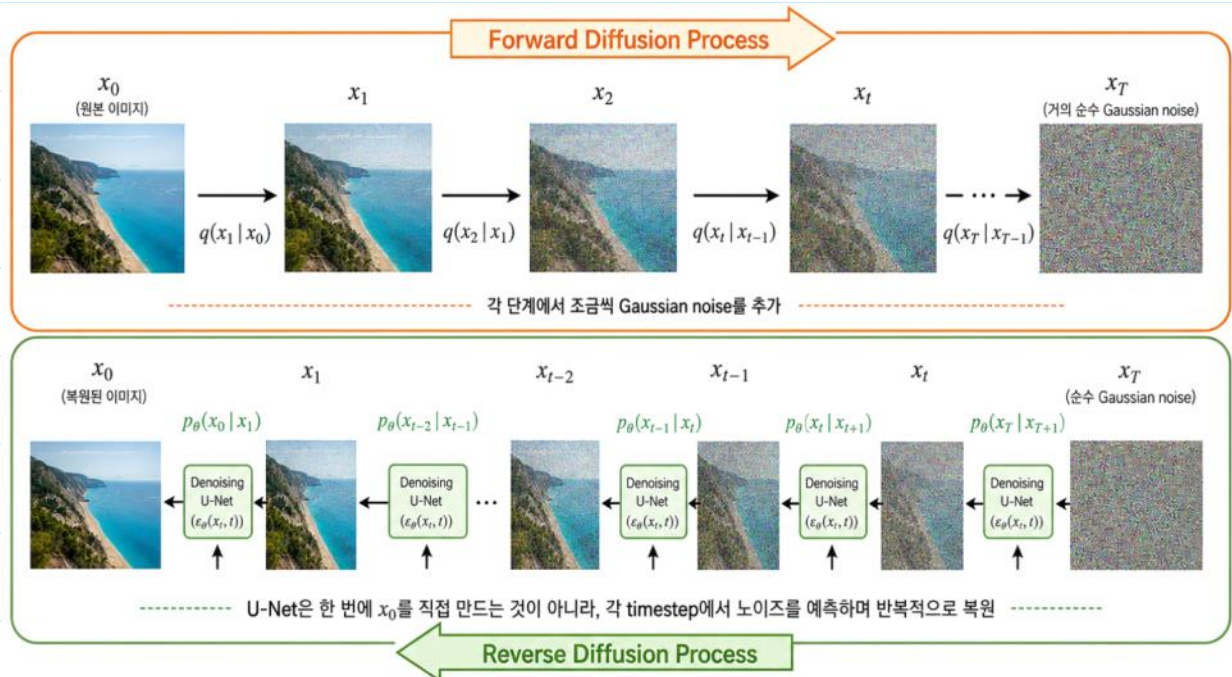
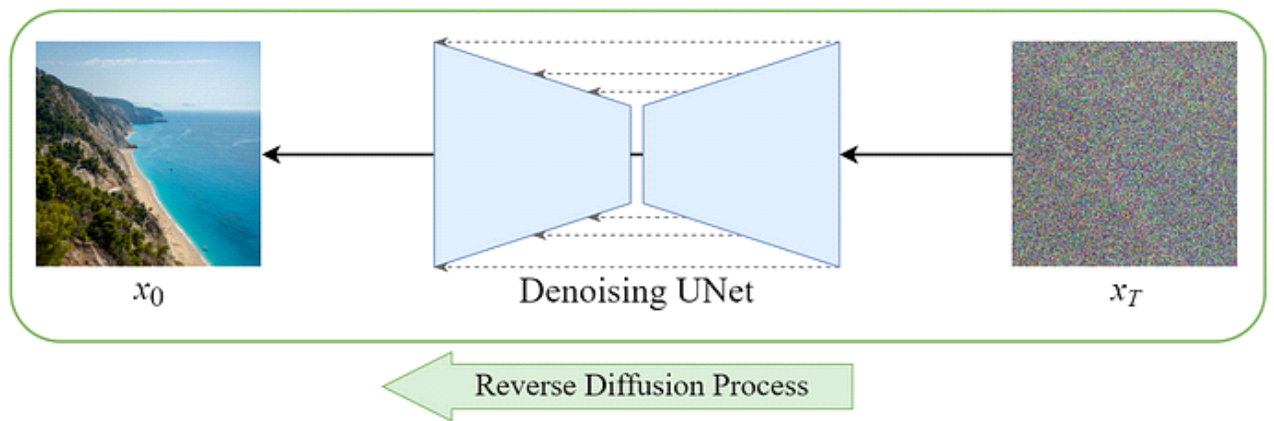
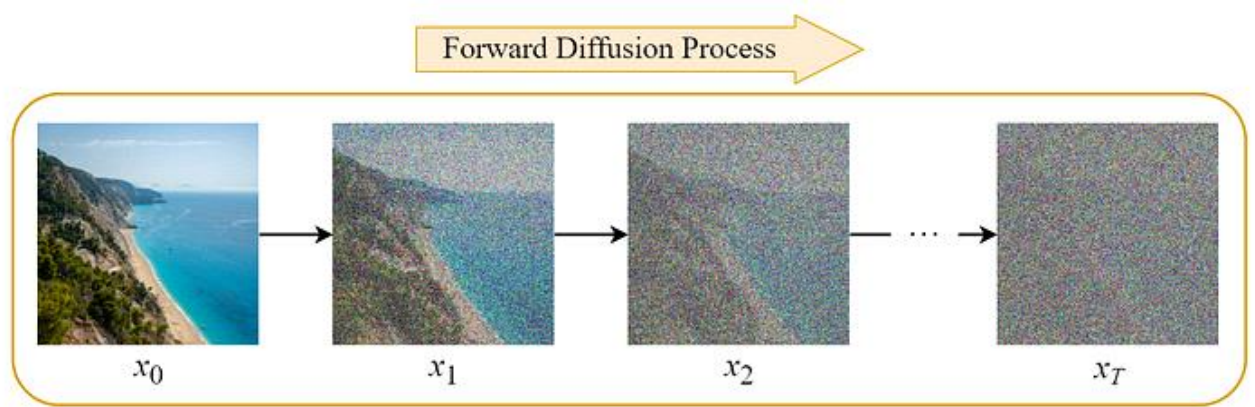


## Diffusion 기반 이미지 생성





## Diffusion 예시 코드

```
def q_sample(x_0, t, alpha_bars):  
  
    noise = torch.randn_like(x_0)  
    sqrt_alpha_bar = alpha_bars[t].sqrt().view(-1, 1, 1, 1)  
    sqrt_one_minus = (1 - alpha_bars[t]).sqrt().view(-1, 1, 1, 1)  
    return sqrt_alpha_bar * x_0 + sqrt_one_minus * noise, noise  
  
T = 1000  
betas = torch.linspace(1e-4, 0.02, T)  
alphas = 1. - betas  
alpha_bars = torch.cumprod(alphas, dim=0).to(device)  
  
model = SimpleUNet(in_channels=4, out_channels=3).to(device)  
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
```

## Diffusion 예시 코드

```
for epoch in range(num_epochs):
    for x_0 in dataloader:
        x_0 = x_0.to(device)

        t = torch.randint(0, T, (x_0.shape[0],), device=device)
        x_t, noise = q_sample(x_0, t, alpha_bars)

        t_norm = t.float() / T
        t_emb = t_norm.view(-1, 1, 1, 1).expand(-1, 1, x_0.size(2), x_0.size(3))

        noise_pred = model(torch.cat([x_t, t_emb], dim=1), t_norm)

        loss = F.mse_loss(noise_pred, noise)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

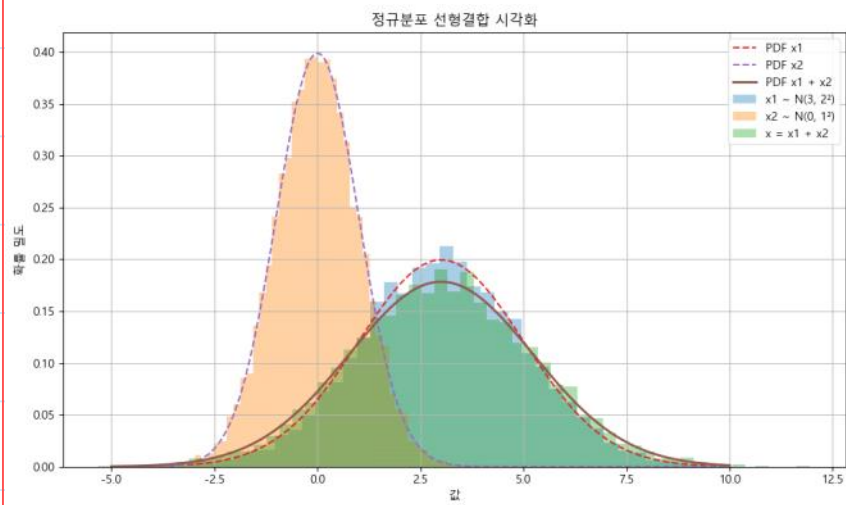
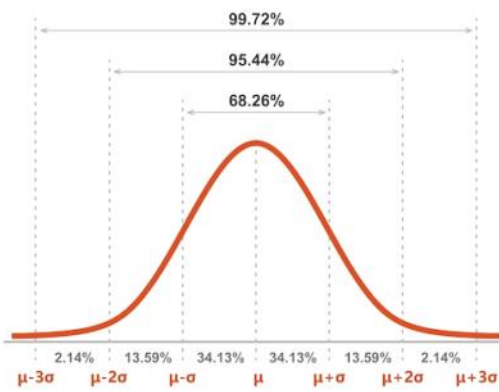
## Diffusion 예시 코드

```
x_t = torch.randn_like(image_shape)

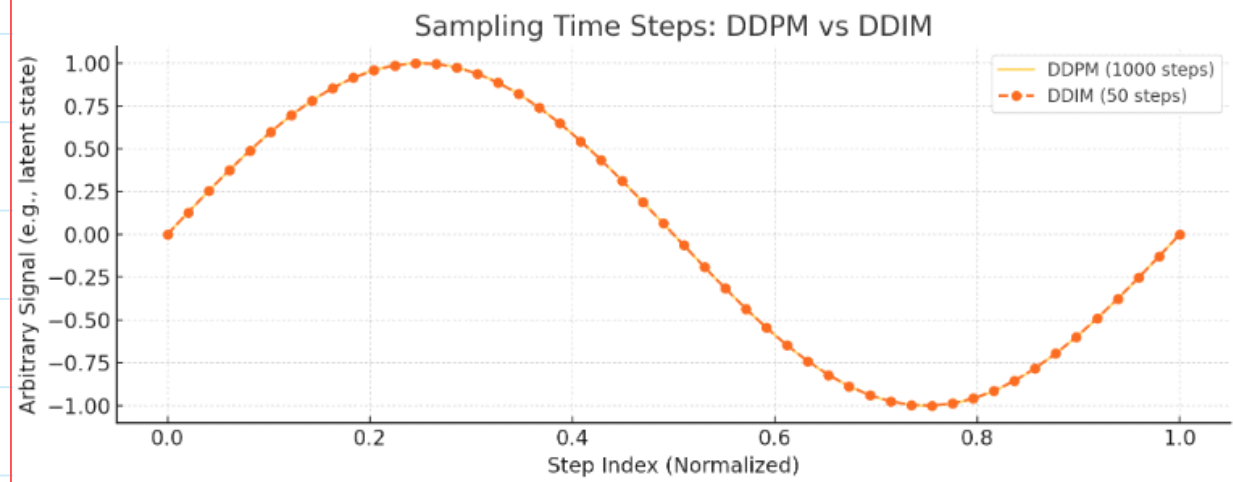
for t in reversed(range(T)):
    epsilon_hat = unet(x_t, t)

    coef1 = 1 / torch.sqrt(alpha_t)
    coef2 = (1 - alpha_t) / torch.sqrt(1 - alpha_bar_t)
    x_prev = coef1 * (x_t - coef2 * epsilon_hat)

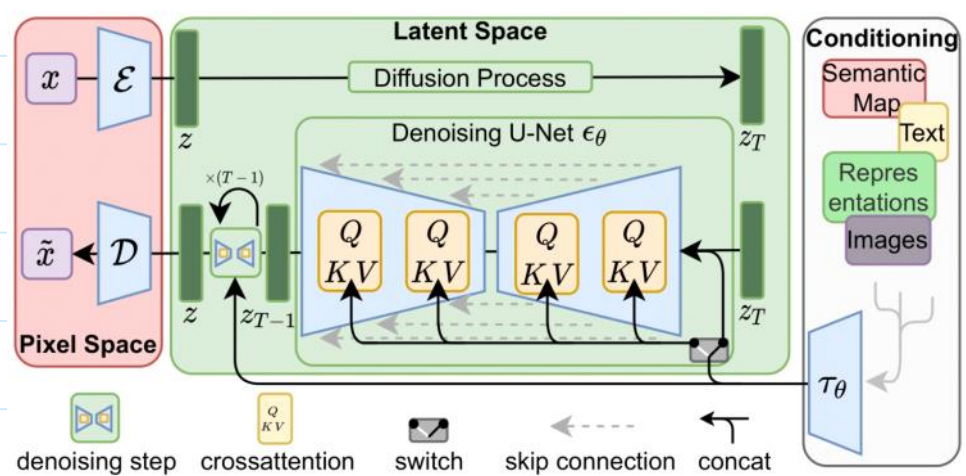
    if t > 0:
        z = torch.randn_like(x_t)
        x_t = x_prev + sigma_t * z
    else:
        x_t = x_prev
```



## DDIM(Denoising Diffusion Implicit Models)



## Latent Diffusion Model (LDM)

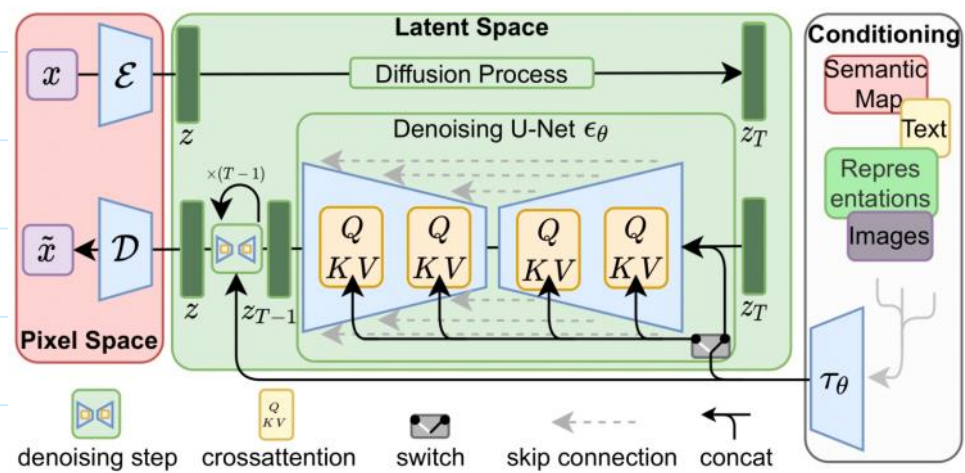


## Stable Diffusion 구성 요소

UNet

VAE(Encoder/Decoder)

Clip Text Encoder



## Stable Diffusion 학습 데이터 세트

LAION (Large-scale Artificial Intelligence Open Network)

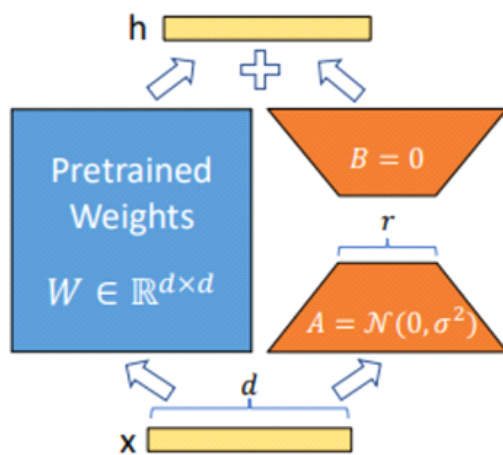
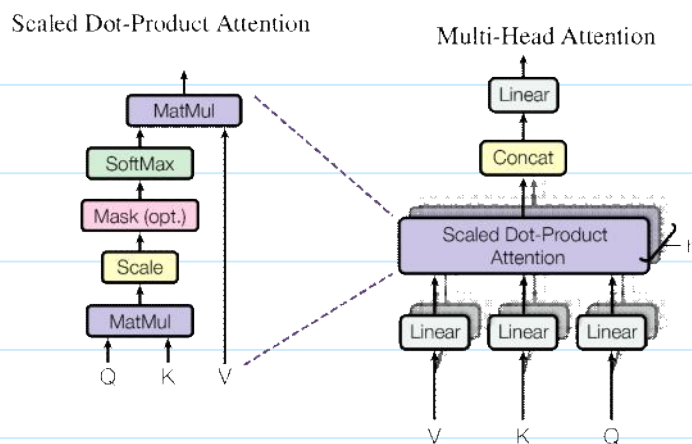


Figure 1: Our reparametrization. We only train  $A$  and  $B$ .



“We find that adding LoRA modules to just the **Query and Value** projections is sufficient.”

## Transfer Learning vs LoRA

| 항목       | LoRA          | 전통적 Fine-tuning<br>(Transfer Learning) |
|----------|---------------|----------------------------------------|
| 가중치 수정   | 기존 weight 고정  | 기존 weight 수정                           |
| 파라미터 수   | 매우 적음 (수천~수만) | 큼 (수백만 이상)                             |
| 기존 성능 유지 | 잘 유지됨         | 파괴 위험 있음                               |
| 활용 대상    | 거대한 모델 일부만 학습 | 전체 또는 특정 모듈 학습                         |

# Stable Diffusion과 의료영상 생성 방법론

## 배경

- Stable Diffusion은 자연 이미지나 예술 그림에는 강하지만, 의료영상 특유의 패턴(CT, MRI, X-ray 등)은 학습된 적이 없거나 적음
- "MRI of a brain"같은 prompt를 넣으면 스타일만 비슷한 가짜 이미지가 생성될 확률 높음
- 진단에 쓰거나 학습 데이터로 쓸 수 있는 정확하고 구조적인 정보 부족

## 해결 전략

### Step 1. LoRA로 의료영상 스타일만 미세 조정

- 기존 Stable Diffusion은 그대로 사용
- 의료영상 10~100장으로 LoRA 모듈 학습
- 학습 속도 빠르고, 기존 모델 보존하며, 적은 데이터로도 시작 가능
- 정확한 병변 위치, 병변간의 경계등을 제대로 이해하기 힘드므로 구조적 표현 한계 존재

### Step 2. VAE 또는 Unet 전체를 Fine-tuning

- 원래 Latent space가 의료에 적합하지 않을 수 있으므로 VAE 수정 필요
- 전체 Unet 구조를 의료영상 기반으로 재학습  
--> 데이터가 수천~수만 장 필요할 수 있음
- 의료영상에 적합한 latent space 생성 및 의료에 특화된 복잡한 구조적인 요소 반영 가능
- 평소 학습할 때 자주 발생하는 여러가지 요소들(overfitting)에 따른 위험성 및 기존 Stable Diffusion의 성능 상실 가능

### Step 3. Text Encoder 조정

- CLIP이 입력한 Prompt를 이해하고, 그에 해당하는 맥락적 의미를 갖는 vector를 생성해야하지만 의료에 대한 텍스트 표현 학습이 부족할 수 있으므로, CLIP을 별도로 의료 텍스트-이미지로 재학습
- CLIP 또한 거대 모델이므로, 잘못 학습할 경우 기존 CLIP의 성능 붕괴 가능성
- 의학적인 표현이 적합한 이미지-텍스트 쌍이 있어야하는데 구하기 어려움